Capability-Bounded Code Evolution with Large Language Models
A Cross-Family Study of Adaptive Heuristic Search

by

Fabio Cozzuto

A dissertation submitted to the
Computer Science
in conformity with the requirements for
the degree of Master of Science

Bishop's University
Canada
June 2026

Abstract

Large language models can now write and revise executable code, which raises a concrete question for heuristic optimization: can an iterative language-model loop generate useful heuristic programs under controlled evaluation, and what limits appear when benchmarks and baselines become more demanding? This thesis studies that question through a unified experimental framework in which models propose deterministic Python heuristics, receive task-specific validator feedback, and are compared under replicated conditions across increasingly realistic problem families. The empirical progression begins with simple adversarial games, then moves to symmetric and asymmetric traveling salesperson benchmarks, and finally to capacitated vehicle routing, including bounded whole-solver evolution on CVRPLIB X instances and a final budget-control closeout on the same bounded CVRP regime. It evaluates replay-aware memory mechanisms, modular operator discovery, adaptive heuristic portfolios, and a cross-family synthesis based on completed experimental campaigns. The evidence supports a capability-bounded interpretation rather than a broad claim of autonomous algorithm discovery. In the game domains, the loop produces measurable behavioral adaptation, but novelty spikes remain uneven and often collapse into churn. In routing, replay effects are family-dependent and mechanism-sensitive: random replay helps on symmetric TSP, compression does not yield a clean win, and no single replay recipe dominates TSP, ATSP, and CVRP simultaneously. In the higher-upside phases, modular operator search generates recurring operator families but no validated reusable operator. Adaptive controller search improves over a weak one-shot LLM selector but not over the strongest fixed or supervised selectors on the primary endpoint. Real-world CVRP solver evolution preserves full held-out feasibility and improves over weaker baselines without surpassing Clarke–Wright savings. The final closeout refines that result: replay-aware incumbent-preserving search outperforms both a budget-matched no-replay control and a bounded direct-generation-plus-repair control, yet still remains behind the strongest fixed Clarke–Wright baseline. It presents a cross-family measurement framework for distinguishing useful adaptation from code churn, benchmark headroom from genuine progress, and bounded heuristic improvement from unsupported claims of general solver discovery.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Dr. Russell Butler, for his guidance, encouragement, and thoughtful feedback throughout this research.

I am also grateful to Bishop's University and Jimmy Couturier for their administrative and financial support.

Scholarships from NSERC (CGRS M) and the Arbour Foundation provided continuity during this project and made it possible for me to carry the work forward.

Finally, I thank my family and friends for their encouragement, patience, and practical help throughout my Master's studies.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

The design of effective heuristics remains one of the central practical problems in computer science. Many computational tasks of scientific and industrial importance, including routing, scheduling, and resource allocation, are too expensive to solve exactly at the scales demanded by real applications. As a result, practitioners routinely rely on hand-crafted heuristics, metaheuristics, or solver portfolios whose quality depends heavily on human insight, empirical tuning, and domain knowledge [25, 5, 8, 12]. The question of how to automate some portion of this design process has therefore persisted across operations research, machine learning, and evolutionary computation for decades.

Recent progress in large language models (LLMs) changes the form of this question. Modern code models can synthesize non-trivial executable programs from natural-language specifications, solve benchmark programming tasks, and improve performance through repeated sampling and automated testing [9, 3, 18, 33, 17, 16]. More importantly for optimization, these models can be embedded inside outer loops that evaluate candidate programs, retain strong variants, and request improved descendants. Systems such as AlphaDev and FunSearch show that evaluator-guided search can go beyond naive one-shot completion when the problem admits fast verification and a suitably structured search space [21, 26]. At the same time, early work on LLMs as evolutionary optimizers suggests both promise and sharp scale limitations, especially when the search is applied to combinatorial optimization rather than small synthetic benchmarks [20, 27].

These developments motivate the present dissertation. The central question is whether repeated proposal, evaluation, and revision can improve executable heuristics in a way that survives stronger benchmarks and stronger baselines. That question also requires tighter controls on interpretation. Apparent gains may reflect useful adaptation, but they may also reflect benchmark headroom, weak baselines, or prompt sequences that generate many variants without preserving a real improvement. This study examines iterative code evolution as an empirical object that must be measured across more than one family and judged by held-out

results rather than by isolated successful artifacts.

1.1 Problem Statement

This thesis studies LLM-guided code evolution as a *measurable adaptive search loop* rather than as a general claim of autonomous algorithm invention. The framework considered here is simple by design. A model receives a constrained programming interface and task feedback, proposes a deterministic Python heuristic, and then receives structured information about the heuristic's behaviour and performance. The loop repeats over many epochs, storing code variants, validator signals, and archive summaries that can be replayed into later prompts. The scientific objective is to determine when such a loop produces held-out improvement and when it merely generates syntactically different but functionally unimportant variants.

The central research question is:

> Under what conditions does an LLM-guided code-evolution loop generate useful heuristic adaptation, and when are apparent gains better explained by search budget, benchmark headroom, validator pressure, or baseline strength?

This broad question is instantiated through several narrower empirical questions distributed across the dissertation:

More specifically, the dissertation asks whether replay-aware memory mechanisms improve held-out performance relative to no replay, random replay, and other replay-selection rules. It also asks whether the answer remains stable when the environment shifts from toy adversarial games to standard benchmark optimization families such as TSPLIB and CVRPLIB, whether the search loop can discover reusable heuristic components or adaptive controllers rather than only full-program variants tied to a single scaffold, and how strongly validator pressure, baseline strength, and benchmark headroom limit the gains that the loop can realize on held-out instances.

1.2 Research Objectives

The project is organized around four concrete objectives.

The objectives are to formalize LLM-guided code evolution as a controlled experimental pipeline rather than an anecdotal prompt-engineering exercise, to evaluate that pipeline on benchmark families whose structure and baseline difficulty differ materially, to distinguish family-specific replay effects from broader claims about memory, search, or benchmark headroom, and to produce a conservative account of the present capabilities and limits of executable heuristic evolution by LLMs, including null results and bounded wins.

1.3 Working Hypotheses

The hypotheses below state the principal empirical claims examined in this dissertation. They are framed to distinguish mechanism-specific effects from broader effects due to search budget, benchmark structure, and baseline strength.

H1: Adaptive search under validator feedback. When a benchmark family contains reachable heuristic headroom and the interface remains executable under repeated sampling, an iterative LLM-guided loop will generate non-trivial behavioural diversity and occasionally improve held-out quality over one-shot sampling.

H2: Replay must justify its added machinery. A replay or archive mechanism should be credited only when it improves held-out performance relative to simpler controls or alternative replay rules evaluated under the same protocol.

H3: Gains shrink under stronger evaluators. Apparent advantages of evolved code should weaken as validator pressure rises, feasibility constraints tighten, and classical baselines consume more of the available benchmark headroom.

H4: Transfer is partial rather than universal. Search components discovered on one family may transfer across related problems, but the strength of that transfer should depend on the interaction structure of the target domain rather than on a task-agnostic notion of "general algorithmic intelligence."

These hypotheses permit negative or mixed outcomes. Under controlled evaluation, null results are informative because they help separate genuine mechanism effects from artefacts of sampling budget or benchmark choice.

1.4 Formal Setup and Notation

The study treats heuristic search as a problem of learning or discovering *programs* that map benchmark instances to feasible solutions. Let $\mathcal{X}$ denote a family of problem instances, let $\Omega(x)$ denote the feasible solution set for instance $x \in \mathcal{X}$, and let $f(\cdot; x)$ denote a minimization objective on $\Omega(x)$.

Definition 1.1. *A* benchmark optimization family *is a triple $\mathcal{F} = (\mathcal{X}, \Omega, f)$ consisting of an instance distribution or benchmark set $\mathcal{X}$, a feasibility map Ω, and an objective function f to be minimized.*

Definition 1.2. *A* deterministic heuristic program *for $\mathcal{F}$ is an executable mapping $h : \mathcal{X} \to \Omega(x) \cup \{\perp\}$ that returns either a feasible solution or a detectable failure symbol $\perp$ under a fixed interface and resource budget.*

This definition covers the constructive game agents of the early phases, the TSP and ATSP tour-construction heuristics of the routing phases, and the bounded CVRP solvers evolved in the later real-world experiments. The relevant distinction is whether a program yields a measurable objective value under controlled execution.

For any evaluation panel $\mathcal{D} \subseteq \mathcal{X}$, the empirical quality of a heuristic program is summarized by an aggregate metric

$$J_{\mathcal{D}}(h) = \frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \mathrm{gap}(h, x), \tag{1.1}$$

where lower values are better and the instance-level gap is defined, whenever a reference cost $f^\star(x)$ is available, by

$$\mathrm{gap}(h, x) = \frac{f(h(x); x) - f^\star(x)}{f^\star(x)}. \tag{1.2}$$

For feasibility-constrained settings such as CVRP, the implementation can replace f by a penalized objective $\tilde{f}$ that incorporates route-capacity or constraint violations. The form of (1.2) is standard in routing benchmarks because it allows comparisons across instances of different scales.

Definition 1.3. *An* LLM-guided code-evolution loop *is an iterative process over a program space $\mathcal{H}$ with prompt state p_t, archive state $\mathcal{A}_t$, and evaluator E, such that*

$$h_t \sim q_\phi(\cdot \mid p_t), \tag{1.3}$$
$$r_t = E(h_t; \mathcal{D}_{\text{train}}), \tag{1.4}$$
$$\mathcal{A}_t = U(\mathcal{A}_{t-1}, h_t, r_t), \tag{1.5}$$
$$p_{t+1} = C(\mathcal{T}, \mathcal{A}_t, r_t), \tag{1.6}$$

where q_ϕ is the model-conditioned program generator, U is the archive update rule, and C constructs the next prompt from the task specification $\mathcal{T}$ and archived evidence.

Equations (1.3)–(1.6) make the research claim precise: replay is not a vague intuition, but a specific modification of the archive state $\mathcal{A}_t$ and prompt-construction operator C. A replay mechanism has independent value only if it improves final held-out performance relative to an explicit control condition. One generic comparison can be expressed as

$$\Delta_{\text{replay}} = J_{\mathcal{D}_{\text{test}}}\left(h^{\text{replay}}\right) - J_{\mathcal{D}_{\text{test}}}\left(h^{\text{control}}\right), \tag{1.7}$$

where a robustly negative value of Δ_{replay} would support a replay-specific advantage on a lower-is-better metric.

To separate meaningful behavioural diversity from mere lexical change, the loop also tracks novelty-like signals. A generic archive-relative novelty score can be written as

$$\nu(h_t, \mathcal{A}_{t-1}) = 1 - \max_{h \in \mathcal{A}_{t-1}} \mathrm{sim}(h_t, h), \tag{1.8}$$

where sim is a code- or behaviour-level similarity measure. No single novelty metric is assumed to be canonical; novelty is treated as a diagnostic that must be interpreted jointly with validator outcomes and held-out quality.

Symbol	Meaning
$\mathcal{X}$	benchmark family or instance space
$x \in \mathcal{X}$	a specific optimization instance
$\Omega(x)$	feasible solution set for instance x
$f(\cdot; x)$	instance-level minimization objective
$J_{\mathcal{D}}(h)$	aggregate evaluation metric of heuristic h on panel $\mathcal{D}$
$\mathcal{H}$	constrained program space searched by the outer loop
q_ϕ	model-conditioned proposal distribution over candidate programs
$\mathcal{A}_t$	archive or replay state available at iteration t
E	validator and scorer used to evaluate a candidate program
$\nu(h_t, \mathcal{A}_{t-1})$	archive-relative novelty diagnostic for candidate h_t

Table 1.1: Core notation used throughout the introduction and later methodological chapters.

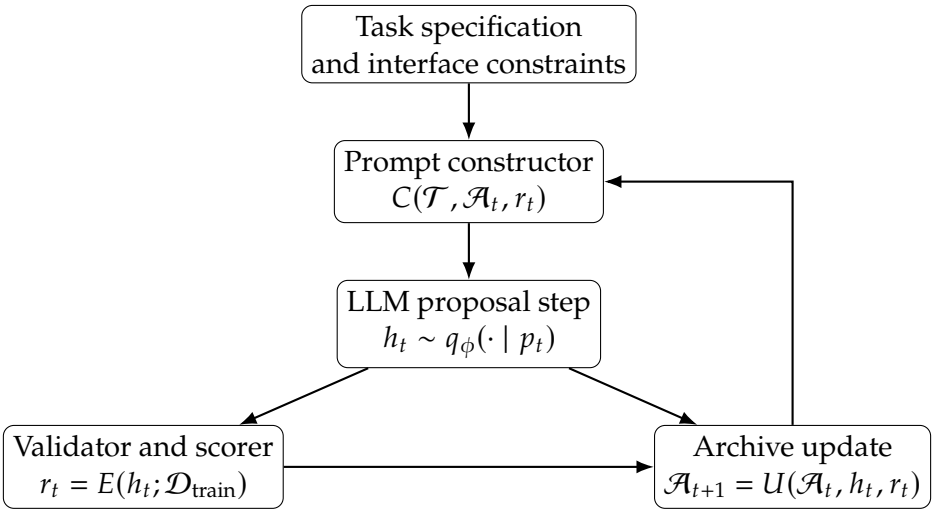

Figure 1.1: Conceptual structure of the LLM-guided code-evolution loop studied in this dissertation. The central question is whether the closed loop between archive, validator, and proposal model produces reliable adaptation under controlled protocols.

1.5 Why This Question Matters

The practical motivation follows directly from the role of heuristics in large-scale optimization. If language models can participate productively in heuristic search, then they could become useful assistants for constructing bounded optimization procedures in domains where exact methods are too slow and hand-designed heuristics are expensive to develop. The scientific motivation is subtler. Recent work on LLM-based scientific and algorithmic discovery has highlighted competitive programming performance, small algorithmic improvements, and evaluator-guided discoveries in specialized settings [18, 21, 26]. Those results are important, but they do not by themselves establish a general understanding of when an LLM-driven search loop should work, when it should fail, and which controls are necessary to distinguish true adaptation from a larger sampling budget.

This issue is especially important in stochastic learning systems, where optimistic conclusions can be driven by best-run anecdotes, fragile benchmarks, or insufficient controls. Empirical machine learning has already seen the cost of such under-specified reporting, particularly in reinforcement learning, where replicated evaluation, uncertainty-aware summaries, and strong baselines are now widely recognized as necessary for credible claims [15, 1]. The same discipline is needed here. A dissertation on LLM-guided heuristic evolution should not only report positive cases. It should also identify limits, null results, and failure mechanisms.

1.6 Dissertation Scope

The work reported here begins with simple adversarial games and then moves to increasingly realistic combinatorial optimization families. This progression is deliberate. The initial game environments provide a cheap setting in which iterative adaptation, curriculum effects, replay, and behavioural diversity can be observed directly. Later phases retain the same basic code-evolution machinery but replace the toy environment with benchmark-driven tasks whose solution quality can be evaluated against established references and classical heuristics. In particular, the dissertation studies adversarial code evolution in simple two-player games, replay-aware heuristic evolution on symmetric TSPLIB traveling salesperson benchmarks, transfer of the same design ideas to asymmetric TSPLIB ATSP and CVRPLIB CVRP families, modular operator discovery and adaptive heuristic portfolio control on TSP, bounded whole-solver evolution on real-world CVRPLIB X instances, a matched-budget closeout on the same bounded CVRP regime comparing replay-aware search against direct synthesis and memoryless no-replay control, and a cross-family synthesis of the completed simple-game, TSP, and real-world CVRP evidence together with a descriptive direct Codex baseline on bounded CVRP.

The study therefore extends beyond a single TSP setting, but it does not attempt to formulate a universal theory of automatic algorithm discovery. Its aim is narrower:

Family	Benchmark basis	Evolved artifact	Primary held-out metric
Simple games	synthetic adversarial environments	deterministic game-playing policy code	win rate / score margin
TSP	TSPLIB symmetric routing instances	tour-construction or improvement heuristic code	relative optimality gap
ATSP	TSPLIB asymmetric routing instances	transfer-tested heuristic code	relative optimality gap
CVRP	CVRPLIB, especially Uchoa X instances	whole solver or route-construction code	penalized gap with feasibility pressure

Table 1.2: Problem families covered by the dissertation and the corresponding evaluation focus.

to develop a unified framework for studying *capabilities and limits* in LLM-guided code evolution across domains with different structural demands.

1.7 Main Thesis

The central thesis of this dissertation is as follows:

> Across simple games, TSP, ATSP, and real-world CVRP, LLM-guided code evolution can generate diverse executable heuristic programs and sometimes preserve held-out improvements, but its success is capability-bounded. In the current evidence, performance depends strongly on validator pressure, baseline strength, benchmark headroom, and task-specific interaction structure. Replay-specific mechanisms can produce a measurable advantage under the right controls, but that advantage is local and benchmark-dependent rather than universal across the completed benchmark families.

The completed results support a conservative interpretation. They do not support two stronger claims: that replay uniformly improves performance, or that iterative LLM search has already surpassed mature human-designed optimization heuristics in a general sense. Instead, the empirical record identifies where adaptation occurs, where it fails to transfer, and how meaningful improvement can be separated from superficial novelty and favorable benchmark conditions.

1.8 Research Contributions

The study makes five main contributions.

It introduces a unified experimental framework for iterative code evolution with archived feedback, constrained execution, deterministic validators, and report-first research artifacts. It extends the study of replay-aware code evolution from

toy adversarial settings to multiple benchmark optimization families, including symmetric TSP, asymmetric TSP, and capacitated vehicle routing. It evaluates not only positive search variants but also mechanism-focused alternatives, including curated failure replay, compression pressure, modular operator evolution, and portfolio-based selection under strong fixed baselines. It emphasizes replicated, uncertainty-aware, benchmark-based evaluation in the spirit of careful empirical RL methodology [15, 1]. Finally, it contributes a conservative cross-family synthesis in which negative and boundary results are treated as findings rather than as failures to be hidden.

1.9 Preview of Findings

The main empirical findings are mixed but coherent.

First, the game phases show measurable functional adaptation, but the novelty audit also shows that many large code changes are mixed or negative rather than uniformly useful. Second, replay can help on benchmark routing, but not in the simple way initially hypothesized. On symmetric TSP, naive raw-failure replay is not the strongest mechanism; curation matters more than failure accumulation alone, and compression does not help the strongest replay arms. Third, the family extensions block any universal replay claim: ATSP yields only a weak compression-aware signal, while no-replay remains strongest on the replicated phase-5 CVRP suite. Fourth, the later higher-upside phases use stronger validators and baselines. Modular operator discovery produces genuine rediscovery structure, yet no operator survives the full validation pipeline. Adaptive portfolio control improves over a weak one-shot LLM selector, yet does not beat the strongest fixed heuristic or the supervised selector when oracle headroom is absent on the primary endpoint. Whole-solver evolution on real-world CVRP stays feasible and improves over weaker bounded baselines, but not over the fixed Clarke–Wright baseline. The final matched-budget closeout then narrows the mechanism claim: replay-aware solver evolution beats both a budget-matched no-replay control and a bounded direct-generation-plus-repair control on held-out penalized gap, yet still remains behind Clarke–Wright. Finally, the cross-family synthesis supports a capability-bounded account in which executable variation is common, but durable gains depend on the target family and on the strength of the competing baseline.

These findings support a *capability-and-limits* interpretation. The completed evidence is sufficient to justify continued study of the framework, but not to justify claims of general algorithm-discovery superiority.

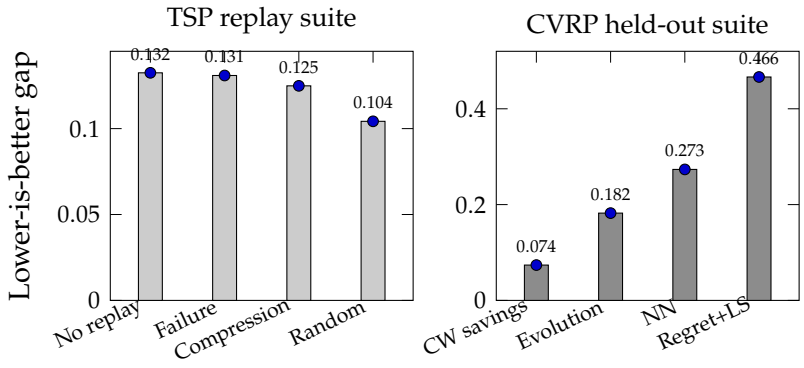

Figure 1.2: Representative held-out results from completed project phases. Lower is better in both panels. In the TSP replay suite, random replay achieved the best mean transfer gap among the replay variants. In the real-world CVRP suite, solver evolution improved over weaker bounded baselines yet remained behind the fixed Clarke–Wright savings heuristic on the primary endpoint.

1.10 Organization of the Dissertation

The remainder of the dissertation is organized as follows. Chapter 2 reviews the literature on heuristics and algorithm selection, machine learning for combinatorial optimization, LLM-based code generation, evaluator-guided algorithm discovery, and curriculum, replay, and diversity mechanisms. Chapter 3 presents the common experimental framework, replication discipline, and reporting methodology. Chapter 4 reports the adversarial transfer evidence and the manual novelty adjudication. Chapter 5 examines the replicated routing replay and replay-mechanism results across TSP, ATSP, and CVRP. Chapter 6 presents the operator-discovery, adaptive-portfolio, and real-world CVRP solver-evolution phases. Chapter 7 returns to the main hypotheses, synthesizes the cross-family evidence, and outlines the principal limitations and future research directions.

Chapter 2

Literature Review

2.1 Heuristics, Hyper-Heuristics, and Algorithm Selection

The practical difficulty of many combinatorial optimization problems has long made heuristic design an essential part of computer science and operations research. Rice's classic formulation of the algorithm selection problem cast the issue in its most general form: given a space of instances, a portfolio of candidate algorithms, and a performance measure, the task is to learn or construct a mapping from instances to algorithms that performs well in expectation [25]. In modern notation, if $\mathcal{X}$ is an instance family, $\mathcal{A}$ is a portfolio of algorithms, and $m(a, x)$ is a lower-is-better performance metric, then the ideal selector is

$$\pi^\star \in \arg\min_{\pi:\mathcal{X}\to\mathcal{A}} \mathbb{E}_{x\sim\mathcal{D}}[m(\pi(x), x)]. \tag{2.1}$$

Equation (2.1) remains foundational because it separates the underlying optimization problem from the meta-level question of which procedure should be applied to which instance.

Hyper-heuristics emerged as one prominent response to this meta-level challenge. Rather than searching directly over candidate solutions, hyper-heuristics search over heuristics or heuristic components, with the goal of automating heuristic selection, sequencing, or generation [8, 12]. This shift from solution space to heuristic space is especially relevant for the present dissertation because the proposed LLM-guided loop also operates primarily at the level of *programs that produce solutions*, not at the level of solutions themselves. Recent surveys emphasize that hyper-heuristics now span selection, generation, portfolio, and configuration settings, and that modern work increasingly integrates machine learning into all four categories [12]. The present dissertation belongs to this tradition, with the difference that the search operator is a language model capable of proposing new executable code rather than a classical mutation, grammar, or rule-combination mechanism.

Algorithm selection has likewise evolved toward both feature-based and feature-free approaches. Traditional methods rely on hand-designed instance descriptors,

whereas newer methods attempt to learn directly from raw instance representations [2]. This distinction becomes important in later chapters, especially for the adaptive heuristic portfolio experiments, because it frames a key design choice: should a controller rely on explicit descriptors crafted from domain knowledge, or can a more generic representation learn the selector implicitly?

2.2 Formal Benchmark Problem Definitions

The empirical phases of this dissertation are built around standard benchmark optimization families. A concise formal account is therefore useful before turning to learning-based methods.

2.2.1 Symmetric Traveling Salesperson Problem

In the symmetric traveling salesperson problem (TSP), one is given a complete weighted graph $G = (V, E, w)$ with $w_{ij} = w_{ji} \geq 0$ and seeks a minimum-cost Hamiltonian cycle [24]. Using binary edge-selection variables x_{ij}, a standard formulation is

$$\min_{x} \quad \sum_{(i,j)\in E} w_{ij} x_{ij} \tag{2.2}$$

$$\text{s.t.} \quad \sum_{j\neq i} x_{ij} = 2, \qquad \forall i \in V, \tag{2.3}$$

$$\sum_{i,j\in S,\ i<j} x_{ij} \leq |S| - 1, \qquad \forall S \subset V,\ 2 \leq |S| < |V|, \tag{2.4}$$

$$x_{ij} \in \{0, 1\}, \qquad \forall (i, j) \in E. \tag{2.5}$$

Constraint (2.3) enforces degree two at every city, while (2.4) removes disconnected subtours. The TSP is NP-hard, which explains why heuristic methods remain central despite decades of exact and approximate progress.

2.2.2 Asymmetric Traveling Salesperson Problem

The asymmetric TSP (ATSP) replaces the undirected graph by a complete directed graph $G = (V, A, c)$ in which c_{ij} and c_{ji} may differ. The goal is again to find a minimum-cost Hamiltonian tour, but now with one incoming and one outgoing arc per city. Many constructive ideas transfer from TSP to ATSP, yet the asymmetry changes the local-search geometry and often weakens heuristics that rely on symmetric exchange structure. This makes ATSP a useful transfer family for testing whether a learned or evolved heuristic reflects a genuine search principle rather than a benchmark-specific shortcut.

2.2.3 Capacitated Vehicle Routing Problem

In the capacitated vehicle routing problem (CVRP), a depot node 0, customer set $V = \{1, \dots, n\}$, customer demands $q_i > 0$, and vehicle capacity Q are given. The task is to partition customers into depot-rooted routes whose total demand does not exceed Q and whose combined travel cost is minimal [10, 31]. A compact route-based view is

$$\min_{\mathcal{R}} \quad \sum_{R \in \mathcal{R}} c(R) \tag{2.6}$$

$$\text{s.t.} \quad \bigcup_{R \in \mathcal{R}} R = V, \qquad R \cap R' = \varnothing \text{ for } R \neq R', \tag{2.7}$$

$$\sum_{i \in R} q_i \leq Q, \qquad \forall R \in \mathcal{R}. \tag{2.8}$$

Unlike TSP, CVRP combines sequencing and packing structure. That added constraint pressure matters for this dissertation because a program that looks promising under tour construction alone may fail once feasibility management, route merging, and penalty handling become part of the evaluator.

2.3 Machine Learning for Combinatorial Optimization

Machine learning for combinatorial optimization has developed into a substantial field in its own right. Bengio, Lodi, and Prouvost provide one of the clearest organizing views: optimization problems can be treated as data points, and learning can be used to replace costly or poorly defined decisions inside larger solving procedures [5]. This framing is useful because it avoids the misleading idea that learning must replace classical optimization wholesale. Instead, machine learning can support branching, ranking, repair, construction, selection, or parameterization within a broader pipeline.

A complementary research stream studies direct learning-based solvers. Neural combinatorial optimization with reinforcement learning showed early promise on Euclidean TSP by training neural policies to construct tours directly from coordinates [4]. Khalil et al. extended this direction by learning greedy algorithms over graph-structured state representations, showing that learned policies can act as reusable meta-algorithms rather than as fixed instance-specific predictors [11]. Subsequent work extended learning-based optimization to more constrained settings and more realistic objectives, but the broader lesson remained the same: learned solvers can be effective, yet strong classical heuristics often remain difficult to beat across wide instance distributions [5]. This tension matters for the current dissertation, which does not ask an LLM to output one final tour directly. Instead, it asks whether an LLM can iteratively improve the *code* of a heuristic solver under benchmark feedback.

The routing literature also provides the baselines and datasets that make such claims meaningful. TSPLIB remains the canonical reference library for traveling salesperson problems [24]. For symmetric TSP, the Lin–Kernighan family and its refinements, including Helsgaun's implementation, remain among the strongest and most influential heuristic baselines [19, 14]. For CVRP, Clarke and Wright's savings heuristic remains a foundational constructive baseline [10], while modern state-of-the-art practice includes far stronger hybrid metaheuristics such as Vidal's hybrid genetic search [32]. The modern Uchoa instance family likewise provides a more challenging and statistically informative benchmark regime than many older CVRP sets [31]. Together, these references define the level of baseline strength against which LLM-evolved heuristics must be judged. A language-model system that only beats naive constructive baselines should not be conflated with one that beats the best established heuristics in the literature.

2.4 Large Language Models for Code Generation

The empirical framework studied in this dissertation depends directly on the rapid improvement of code-capable language models. Codex demonstrated that large language models trained on code could solve a meaningful fraction of functional programming tasks and that repeated sampling substantially boosts success rates on harder problems [9]. Austin et al. likewise showed that program synthesis performance scales strongly with model size and that natural-language feedback can materially improve generated programs [3]. These findings indicate that outer-loop search and repair are integral components of strong performance on demanding code-generation tasks.

As the field matured, surveys began to distinguish between simple code completion, benchmarked program synthesis, repair, debugging, and agentic multi-step workflows. Early overviews of NL2Code models emphasized training data, scaling, and benchmark design [33], while more recent surveys document a much larger and more heterogeneous landscape of code models, evaluation protocols, and practical risks [17]. At the same time, stronger benchmarks such as AlphaCode-style competitive programming and LiveCodeBench reveal that the difference between solving simple textbook-style tasks and solving harder, more recent coding problems remains substantial [18, 16]. This distinction reinforces the need to evaluate code generation under task-specific execution and held-out tests, rather than by surface plausibility alone.

2.5 Evaluator-Guided Search and Algorithm Discovery

One-shot code generation is only one end of the design spectrum. A more relevant line of work for this dissertation combines generative models with evaluators that

score candidate programs and feed the results back into an outer search loop. In abstract form, the objective is no longer to predict source code that merely "looks right," but to search a program space $\mathcal{H}$ for

$$h^\star \in \arg\min_{h \in \mathcal{H}} J_{\mathcal{D}}(h), \tag{2.9}$$

where $J_{\mathcal{D}}$ is defined by measurable performance on a benchmark panel. This framing is important because it aligns the generative model with verification rather than with free-form textual plausibility.

FunSearch is a central example. It pairs a frozen LLM with a systematic evaluator and a diverse program archive in order to discover high-scoring programs for problems that are hard to solve but easy to evaluate [26]. Its importance lies not only in its results, but in its formulation: the LLM is used as a variation operator inside an evolutionary process, while the evaluator guards against incorrect or hallucinated proposals.

AlphaDev demonstrates a related idea from a reinforcement-learning perspective. It formulates algorithm discovery for low-level sorting routines as a single-player game and optimizes directly for correctness and latency at the assembly level [21]. AlphaCode, while not an evolutionary system in the same sense, also depends heavily on large-scale sampling, behaviour-based filtering, and selection among many candidate programs [18]. These systems show that modern code-generating AI often relies on the interaction between generative diversity and external verification, not on a single perfect sample.

The most direct predecessor to the present dissertation in combinatorial optimization is the study of LLMs as evolutionary optimizers by Liu et al. [20]. Their system treats the LLM as a crossover-and-mutation operator within a population-based optimizer and shows promising TSP results on very small instances. This work is conceptually important, but its scale also exposes a gap. There remains a need for research that studies executable whole-program heuristic evolution on standard benchmark families, with stronger baselines, repeated trials, transfer evaluation, and mechanism controls. A recent systematic review of LLMs in combinatorial optimization confirms both the rapid expansion of the area and the diversity of roles LLMs now play in optimization systems [27]. However, the review also implies that careful cross-family empirical evaluation remains comparatively rare.

2.6 Curriculum, Replay, and Diversity Mechanisms

The replay-aware components of this dissertation draw from several adjacent literatures rather than from one single source. Curriculum learning offers one motivation. Bengio et al. formalized the idea that ordering examples from easier to harder can improve learning and generalization [6]. Later surveys and automated curriculum methods extended the idea to a wide range of machine learning and

reinforcement learning settings [30, 13]. In adversarial or sequential domains, curricula matter because the learner's exposure distribution is part of the effective training algorithm.

Replay mechanisms supply a second motivation. In reinforcement learning, prioritized experience replay showed that non-uniform reuse of past transitions can improve learning efficiency relative to uniform replay [28]. In one common formulation, transition i is drawn with probability

$$P(i) = \frac{p_i^\alpha}{\sum_j p_j^\alpha}, \tag{2.10}$$

where p_i is a priority score and α controls the strength of prioritization. Later work generalized this idea to sequence-level replay and more structured replay decisions [7]. Although the present dissertation does not replay transitions in the standard RL sense, it reuses *instances*, failure cases, and archive summaries as a memory mechanism for future program proposals. That analogy is strong enough to be informative, but weak enough that it must be tested rather than assumed. A replay strategy that helps temporal-difference learning does not automatically imply a successful replay strategy for program-level heuristic evolution.

Diversity-based search provides a third motivation. Quality-diversity methods such as MAP-Elites and novelty-search-with-local-competition aim not merely to optimize a single objective, but to retain a collection of high-performing, behaviourally distinct artifacts [22, 23]. This family of ideas is relevant because a code-evolution loop can easily collapse to narrow local search unless the archive preserves multiple useful regions of the behavioural space. The present dissertation therefore draws on QD-style intuitions when it tracks novelty, maintains archives, and evaluates whether diversity corresponds to genuine adaptation rather than lexical churn.

Finally, self-play and adversarial practice provide a motivating example for why curriculum and iterative pressure can induce capability growth at all. AlphaGo Zero showed that high-level performance can emerge from repeated self-play without human demonstrations [29]. Its scale and claims differ substantially from those considered here. Nevertheless, the underlying intuition remains relevant: iterative interaction can sometimes unlock behaviour that is hard to obtain from static supervision alone.

2.7 Methodological Discipline and the Need for Conservative Claims

An important part of the literature relevant to this dissertation concerns *how* claims are validated, not only *what* systems are built. Henderson et al. document the reproducibility problems that arise when stochastic learning systems are evaluated with too few seeds, weak statistical reporting, or poorly matched baselines [15].

Agarwal et al. extend this point by arguing for robust statistical summaries and uncertainty-aware comparisons in deep RL benchmarking [1]. Although these works target reinforcement learning rather than code evolution, the methodological lesson transfers directly. When a generative system can produce very different candidate programs across runs, best-case anecdotes are not enough.

This methodological literature motivates the use of paired offsets, aggregate reports, bootstrap comparisons, held-out panels, and mechanism-specific controls in the present study. A simple paired comparison between two conditions A and B can be written as

$$\hat{\Delta}_{A,B} = \frac{1}{n} \sum_{i=1}^{n} \left(m_i^{(A)} - m_i^{(B)} \right), \tag{2.11}$$

with uncertainty estimated by resampling or other repeated-evaluation procedures. A negative result obtained under proper controls can be more informative than a seemingly positive result that confounds memory mechanisms with extra candidate budget.

2.8 Synthesis and Research Gap

Table 2.1 summarizes the main research traditions from which this dissertation draws.

The literature reviewed above supports five observations that matter directly for this thesis.

Classical optimization already provides strong benchmark libraries and strong heuristic baselines [24, 19, 14, 10, 31, 32]. Hyper-heuristics and algorithm selection offer established ways to reason about search in heuristic space rather than solution space [25, 8, 12]. Modern code LLMs are strong enough to generate executable programs, especially when combined with repeated sampling, testing, and repair [9, 3, 18, 16]. Evaluator-guided algorithm discovery with AI is now credible, but existing success stories often depend on specialized evaluators, narrow scopes, or problem formulations that differ from full-program benchmark optimization [21, 26, 20]. Finally, curriculum, replay, and diversity mechanisms are theoretically motivated, but their value in LLM-guided *code evolution* remains an empirical question rather than a settled fact [6, 28, 23].

This thesis addresses a gap at the intersection of these points. There is still no clear empirical account of how LLM-guided code evolution behaves across multiple benchmark families when evaluated with replicated runs, strong classical baselines, replay alternatives, and strict validator-based transfer tests. The study therefore centers on a conservative cross-family study of executable heuristics rather than on isolated positive examples.

Tradition	Search object	Main strength	Main limitation	Representative refs.
Classical heuristics and metaheuristics	solutions or route moves	strong benchmarked performance	expensive domain expertise	[19, 14, 10, 32]
Hyper-heuristics and algorithm selection	heuristic choices or generators	explicit heuristic-level reasoning	relies on crafted operators or features	[25, 8, 12]
Learning-based combinatorial optimization	policies, rankings, or solver components	learns recurring structure from data	difficult to beat strong classical solvers broadly	[5, 4, 11]
Evaluator-guided program search with LLMs	executable programs	open-ended code generation plus verification	noisy search; mechanism claims need strong controls	[26, 21, 20]

Table 2.1: Positioning of the present dissertation relative to adjacent literatures. The thesis sits closest to evaluator-guided program search, but its experimental logic also depends on benchmarked routing, hyper-heuristic reasoning, and conservative empirical methodology.

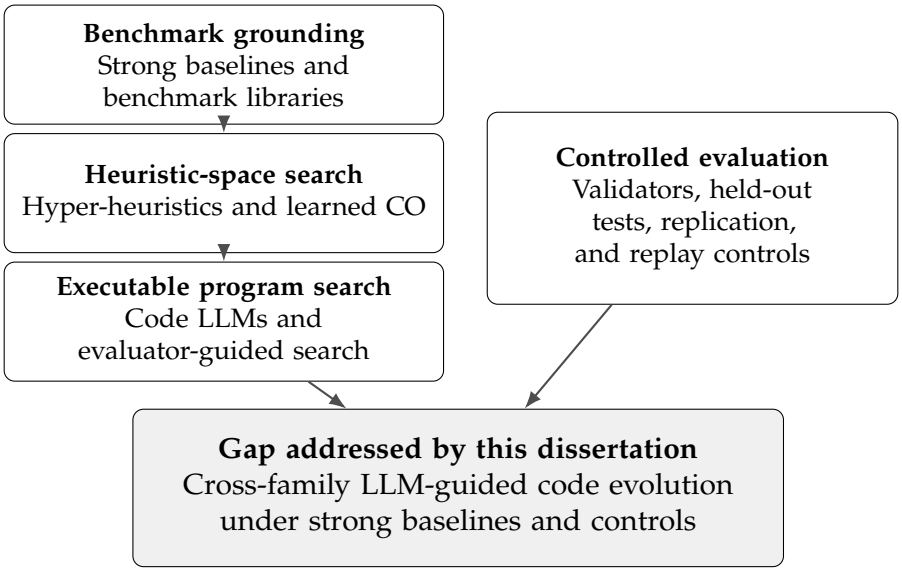

Figure 2.1: Conceptual synthesis of the research threads integrated by the dissertation. The gap is not any single ingredient in isolation, but the absence of a cross-family account that combines benchmark grounding, heuristic-space search, executable-program generation, and strict validator-based evaluation in one empirical framework.

2.9 Chapter Summary

The literature review established the technical context for the dissertation in four steps. It positioned the work within algorithm selection and hyper-heuristics, where the central object of search is a heuristic procedure rather than a single candidate solution. It reviewed the benchmark optimization families that make the empirical claims meaningful, including the standard TSP, ATSP, and CVRP formulations and their mature baseline literatures. It also situated the project within modern code-generation and evaluator-guided program-search research, where LLMs are most credible when paired with strong external verification. Finally, it showed why replay, curriculum, and diversity mechanisms must be evaluated under careful experimental discipline rather than inferred from isolated positive cases. The next chapter turns from that context to the experimental design used to study the gap identified here.

Chapter 3

Experimental Methodology

This chapter describes the common experimental design used across the completed phases of the project. It defines the evaluation framework carried from simple adversarial games to benchmark routing and, finally, to bounded real-world vehicle routing. The framework is deliberately conservative. Each phase relies on deterministic validators, held-out evaluation panels, repeated runs, and phase-specific technical validity checks before any substantive interpretation is accepted.

3.1 Research Design Overview

This study reports a sequence of completed experimental campaigns rather than a single monolithic study. The sequence was chosen so that the search object, benchmark structure, and validator pressure become progressively more demanding. Early phases test whether iterative code evolution can alter behavior at all. Later phases ask whether the same loop can preserve useful adaptations on standard optimization benchmarks with stronger baselines and clearer notions of feasibility.

Table 3.1 summarizes the completed evidence base retained in the final dissertation.

The archival phase record is finer-grained than the bundled presentation used in the dissertation. In the project archive, phases 1, 2, 2b, 3, and 4 are preserved as distinct simple-game campaigns. The final manuscript groups them into one simple-game evidence block because they share the same environment family and lead to one retained transfer-and-novelty conclusion. The phase-9 closeout is retained separately because it is not a new task family, but a scoped mechanism study on the same bounded CVRP regime under matched learned controls. Phase 10 is retained as a synthesis stage rather than as a new benchmark family. A separate phase-11 model-strength factorial study is preserved in the research archive, but it is not admitted into the retained evidence base because the final dissertation does not rely on that scoped follow-up campaign.

Phase	Search object	Replicates	Conditions	Primary endpoint
1–4 (simple games)	full deterministic game-policy code	10 transfer runs plus manual review	3 transfer environments; 3 audited curriculum recipes	held-out win rate and score margin
5 (routing replay transfer)	constrained routing heuristic code	20 paired offsets per family	4 replay conditions across TSP, ATSP, and CVRP	held-out benchmark optimality gap
6 (replay mechanism study)	constrained TSP heuristic code	20 paired offsets	7 replay variants	held-out TSPLIB optimality gap
7 (operator discovery)	modular TSP operator code	20 paired offsets	7 conditions	held-out TSPLIB gap plus transplant/ablation validation
8 (adaptive portfolio)	TSP controller over a frozen heuristic portfolio	20 paired offsets	8 conditions	held-out TSPLIB gap and selector regret
9 (real-world solver evolution)	bounded CVRP solver code	20 paired offsets	4 conditions	held-out penalized gap with feasibility pressure
9 closeout (budget control)	bounded CVRP solver code under matched budgets	20 paired offsets	6 conditions	held-out penalized gap with explicit learned controls
10 (cross-family synthesis)	family-level result summaries	archived official artifacts	completed simple games, routing, and bounded CVRP plus 1 descriptive direct Codex baseline	normalized interpretation rather than pooled win counting

Table 3.1: Completed empirical phases discussed in the final dissertation. A crossed model-tier study was scoped but is treated here as future work rather than as part of the completed evidence base.

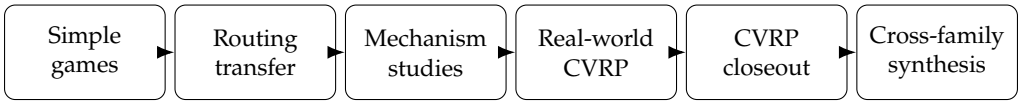

Figure 3.1: Progression of the completed experimental program, from low-cost game adaptation to routing, mechanism studies, higher-level search objects, bounded real-world CVRP, and the final matched-budget closeout on the same CVRP regime. Portfolio control is included in the mechanism-study stage.

3.2 Common Code-Evolution Loop

All phases instantiate the code-evolution loop introduced in Section 1.4. At each epoch, the model receives a bounded interface, the current incumbent or scaffold, and structured feedback from prior evaluations. It then proposes a deterministic Python artifact: a game policy, routing heuristic, modular operator, controller, or whole CVRP solver. That proposal is validated, executed on the training panel, and either rejected or admitted into the evolving archive.

Several constraints are shared across families.

Across all families, generated code is deterministic and auditable: imports, hidden file or network access, and arbitrary external solver calls are disallowed. Task-specific validators separate syntactic success from scientific usefulness, so a candidate must execute and return a structurally valid output before it can be considered for acceptance. Held-out evidence remains outside the inner loop; training probes, replay probes, and archive summaries may influence search, but the final claims are based on frozen end-of-run evaluation on held-out panels. Fallback or errored generations are logged diagnostically rather than being silently counted as successful improvements.

These constraints matter because they narrow the interpretation of success. A generated program is useful only when it survives validation, remains executable across repeated evaluations, and improves held-out performance under the benchmark-specific metric.

3.3 Family-Specific Search Objects and Benchmarks

The shared loop is implemented differently across families because the scientific question changes with the object of search.

In the simple-game phases, the search object is full policy code for a two-player environment. The environments emphasize transfer and behavioral change rather than exact optimality. Held-out win rate and score margin are therefore the relevant outcomes.

In the routing replay phases, the search object is a constrained heuristic scaffold. For symmetric TSP and ATSP, the evolved code modifies bounded construction, perturbation, and local-search behavior over benchmark instances derived from TSPLIB [24, 19, 14]. In the initial CVRP extension, the same replay logic is transferred to a constrained multi-route scaffold evaluated on CVRPLIB-style instances [10, 31]. Relative or penalized gap is the primary metric because it standardizes performance across instances of different scales.

Phase 7 changes the search object again, from whole-solvers to *modular operators*. The aim is not simply to find a better full heuristic, but to ask whether a reusable perturbation, candidate-pruning, or restart component can survive transplant and ablation checks. Phase 8 then switches from operator evolution to *algorithm selection*:

the model evolves a bounded, interpretable controller over a frozen portfolio of classical TSP heuristics, which places the phase directly within the algorithm-selection tradition introduced by Rice [25]. Phase 9 moves to bounded whole-solver evolution on real-world CVRPLIB X instances [31], where feasibility, repair logic, and runtime variability matter as much as route cost. The phase-9 closeout then keeps the same bounded CVRP regime and validator, but replaces the original comparison set with matched learned controls designed to distinguish replay-specific value from extra candidate budget or direct one-repair synthesis alone.

3.4 Replication, Aggregation, and Statistical Reporting

The reporting discipline follows the conservative empirical guidance discussed in Section 2.7. The routing, operator, portfolio, and real-world CVRP phases were all run with fixed paired seed offsets, usually twenty offsets shared across compared conditions. This design supports paired deltas of the form in Equation (2.11) and reduces the chance that one condition appears stronger only because it benefited from an easier stochastic draw.

The aggregate reports emphasize four summary types.

The aggregate reports therefore emphasize condition means on the primary held-out endpoint, paired condition deltas with 95% bootstrap intervals, technical-validity summaries including fallback counts, generation errors, timeout checks, and acceptance counts, and diagnostic metrics such as code novelty, adaptation efficiency, archive hardness, or runtime inflation when those metrics are relevant to the specific phase.

The study does not rely on best-run anecdotes. When a phase produces a strong positive result, that result is reported together with the paired uncertainty and with the corresponding validity checks. When a phase produces a null or boundary result, that outcome is treated as a finding rather than as a failed attempt to be hidden.

3.5 Qualitative Audit and Validity Safeguards

Not every important question can be answered by scalar metrics alone. Two additional safeguards are therefore used.

First, the simple-game transfer work includes a manual novelty adjudication pass. The largest code novelty spikes are reviewed and classified as functional adaptation, mixed or local adjustment, or churn/failed adaptation. This guards against the common mistake of equating large lexical change with strategically meaningful change.

Second, the later routing phases use phase-specific technical validity audits before interpretation. Examples include checking that all paired runs and all

conditions are present, verifying that accepted artifacts did not depend on fallback behavior, and confirming that operator or solver validation pipelines executed as intended. The phase-7 operator campaign, for instance, was deemed ready for interpretation only after confirming zero modular generation fallbacks and zero materialization fallbacks in the official run set. Phase 9 was likewise interpreted only after checking full held-out feasibility, zero accepted fallback generations, and the absence of hidden solver-worker timeout contamination in the completed official runs. The phase-9 closeout adds one more safeguard to the same regime: replay is credited only after comparison against matched learned controls that remove the simpler explanations of extra search budget or fresh direct synthesis.

3.6 Scope of the Retained Evidence

The final manuscript is intentionally restricted to completed phases whose evidence is coherent and whose reporting pipeline is already established. One descriptive exception is retained: a direct single-shot Codex-authored CVRP solver evaluated through the same phase-9 validator. That artifact serves as a sanity baseline for interactive coding assistance, but it is not pooled with the replicated stochastic campaigns because it is neither randomized nor generated under the same frozen API-loop protocol.

This scope choice reflects the dissertation's broader methodological position. The objective is not to maximize the number of claimed wins, but to produce a defensible account of when the code evolution loop adapts, when it churns, and how those patterns change as validators and baselines become harder.

For reproducibility, the retained claims in this dissertation are tied to official archived run artifacts and code snapshots preserved in the project version-control record. Where stable versioned snapshots exist, such as phase-completion tags, those archived states are the intended reference points for the retained evidence rather than a moving development branch.

Chapter 4

Adversarial Transfer and Early Behavioral Evidence

The simple-game phases serve two purposes in the dissertation. First, they provide a low-cost environment in which iterative code evolution can be observed over many epochs without the computational burden of large benchmark routing suites. Second, they make it possible to inspect the relationship between code novelty and *behavioral* change before moving to harder optimization domains. The results from these phases do not establish routing-quality claims, but they do establish whether the loop can produce transfer-relevant adaptation at all.

4.1 Transfer Setting

The retained simple-game evidence combines two artifacts: the official transfer suite and the phase-4 novelty adjudication. The transfer suite spans ten replicated runs, three transfer environments, and 3000 total epochs. The environments are pursuit/evasion, resource collection/denial, and territory control.

The purpose of the suite is not to reopen the earlier curriculum search, but to evaluate whether the evolved agents remain effective when the environment changes and when performance is measured on held-out opponent panels. The main quantitative outcomes are hold-out win rate and hold-out score margin.

The novelty adjudication provides the complementary qualitative control. It reviews the strongest novelty spikes from three transfer-era curriculum recipes: *rotating-opponents holdout endpoint*, *rotating + nemesis + novelty + replay*, and *rotating + replay-aware selection*.

Each reviewed spike is conservatively classified as functional adaptation, mixed or local adjustment, or churn/failed adaptation. That distinction matters because a transfer claim in the early phases should refer to actual strategy change, not only to different code text.

Recipe	Functional	Mixed/local	Churn/failed
Rotating-opponents holdout endpoint	2/10	2/10	6/10
Rotating + nemesis + novelty + replay	5/10	2/10	3/10
Rotating + replay-aware selection	5/10	2/10	3/10

Table 4.1: Manual adjudication of the strongest novelty spikes in the phase-4 transfer-era recipes, with ten audited spikes per recipe. The heavier replay-aware recipes show more genuine functional adaptations than the simple rotating baseline, but substantial churn remains even in the stronger recipes.

4.2 Manual Novelty Adjudication

Table 4.1 summarizes the manual calls for the top novelty spikes in each recipe.

Three points follow from Table 4.1. First, the baseline rotating-opponent recipe is dominated by churn or failed adaptation among its largest code changes. Second, the heavier replay-aware recipes do improve the ratio of useful to useless novelty spikes, which is evidence that added selection pressure can matter. Third, the stronger recipes still do not justify a claim of broad strategic invention. Even when they produce functional improvements, those improvements are uneven across environments and remain mixed with several large but unhelpful code changes.

4.3 Held-Out Transfer Performance

The official cross-family synthesis reports the family-level hold-out performance shown in Figure 4.1. The numbers are aggregated at the environment level because the simple-game role in the dissertation is to establish transfer-capable adaptation, not to defend one specific curriculum recipe as universally optimal.

The three environments differ materially. Territory control shows the clearest success signal, with hold-out win rate 0.9467 and mean score margin 34.17. Pursuit/evasion is weaker but still positive, with win rate 0.70 and margin 3.702. Resource-collection/denial is the most difficult transfer environment, with win rate only 0.484 despite a positive mean margin of 1.348.

This pattern is consistent with the novelty audit. The most convincing functional adaptations were concentrated in the territorial environment, while the other environments showed a larger share of local retuning and churn. The simple-game evidence therefore supports a bounded positive claim: iterative code evolution can produce real transfer-relevant behavioral change, but that change is domain-dependent and cannot be inferred from novelty alone. Appendix A.1 and Appendix A.2 provide representative accepted and rejected artifacts from these early phases.

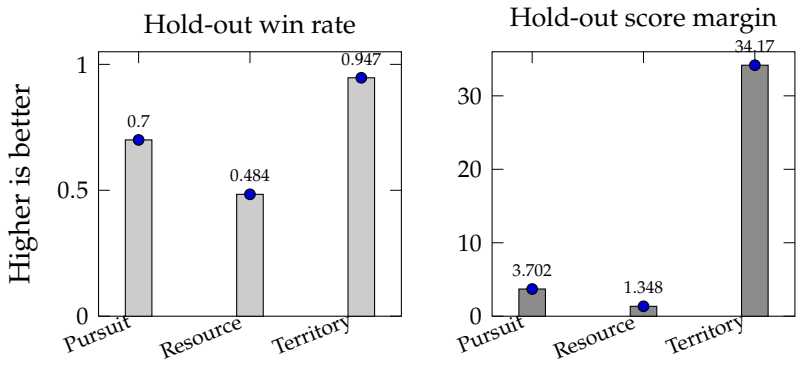

Figure 4.1: Family-level held-out outcomes for the simple-game transfer suite. Territory control shows the strongest positive transfer, pursuit/evasion remains moderately positive, and resource-collection/denial is the weakest environment.

4.4 Interpretation

The simple-game phases support the dissertation in three ways.

First, they provide direct evidence for Hypothesis H1 from Section 1.3. The loop does generate executable diversity, and some of that diversity corresponds to useful held-out adaptation rather than to random code motion.

Second, they establish an important caution that remains true in later chapters: lexical novelty is not a reliable proxy for functional gain. Without the manual adjudication pass, the early project could easily have overstated the meaning of large code changes.

Third, the game phases motivate the move to routing benchmarks. Once the possibility of adaptation is established in a low-cost environment, the harder question becomes whether the same machinery survives stronger baselines, clearer notions of feasibility, and more standardized benchmark evaluation. The next chapter addresses that question directly through replicated TSP, ATSP, and CVRP studies.

Chapter 5

Routing Replay and Mechanism Studies

The routing phases move the project from behavioral game environments to benchmark optimization. This transition matters because routing benchmarks provide stronger baselines, fixed held-out panels, and more standardized notions of progress. The original routing hypothesis was that replay-aware code evolution would improve held-out benchmark gap, and that the improvement would be strongest when replay focused on informative failures. The completed evidence is more nuanced.

5.1 Phase-5 Routing Transfer Across TSP, ATSP, and CVRP

The first routing campaign compares four replay conditions across three benchmark families: no replay, random replay, raw-gap failure replay, and failure replay with compression pressure.

Each family uses twenty paired offsets and reports paired bootstrap intervals on the relevant held out benchmark endpoint. The resulting evidence is summarized in Table 5.1.

Table 5.1 rules out a simple replay-dominance claim. Replay does help on symmetric TSP, but the strongest arm is random replay rather than raw failure replay. ATSP retains only a weak compression-aware signal, and the primary held-out ATSP endpoint is not cleanly separated. In the replicated phase-5 CVRP suite, no-replay remains the best overall condition.

The aggregate summaries establish the direction of the family-level effects, but the offset-level paired deltas show how uneven those effects remain. Figure 5.1 displays the paired held-out deltas for the two phase-5 contrasts that matter most to the later interpretation: random replay versus no replay on symmetric TSP, and compression-aware failure replay versus no replay on ATSP.

For the thesis interpretation, this distinction matters. If replay had behaved

Family	Best mean condition	Key paired comparison	95% interval	Interpretation
TSP	random replay	held-out TSPLIB delta versus no replay: −0.0311; combined transfer delta: −0.0282	[−0.0602, −0.0008] on TSPLIB; [−0.0520, −0.0029] on combined transfer	clear positive replay signal, but the winning arm is random rather than failure replay
ATSP	failure replay with compression	held-out ATSP delta versus no replay: −0.0056; combined transfer delta: −0.0063	[−0.0191, 0.0071] on ATSP; [−0.0133, −0.0002] on combined transfer	weakly favorable, not decisively separated on the primary endpoint
CVRP	no replay	failure replay versus random replay on held-out CVRPLIB: −0.0076	[−0.0146, −0.0006]	replay is not dominant; no-replay remains the strongest family-level baseline

Table 5.1: Phase-5 family summaries across the first replicated routing campaign, based on twenty paired offsets per family. Lower deltas are better because all routing endpoints are lower-is-better gap metrics.

uniformly across TSP, ATSP, and CVRP, then the dissertation could argue for a reusable memory mechanism with broad routing scope. The evidence is narrower: replay pressure interacts with benchmark structure, and the initially favored raw-failure recipe is not the stable winner even within routing. A representative replay-generated TSP heuristic fragment is shown in Appendix A.3.

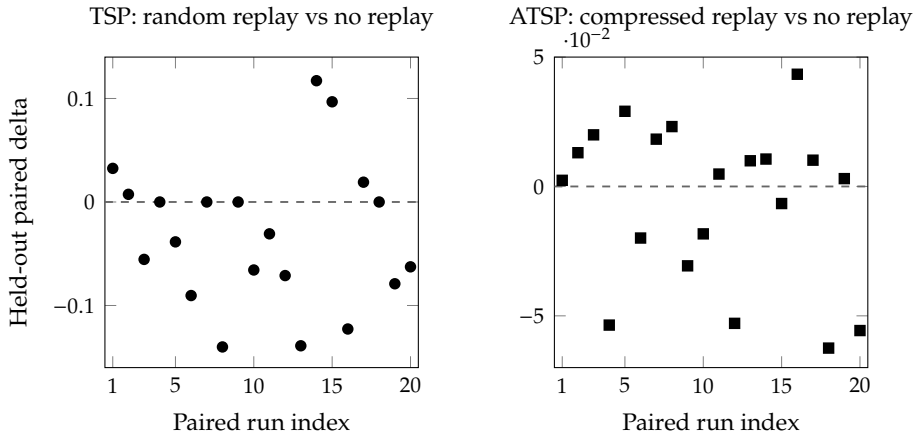

Figure 5.1: Per-run paired held-out deltas for the two phase-5 replay contrasts most relevant to the later thesis interpretation, using twenty matched offsets in each family. Negative values are better because the endpoint is lower-is-better gap. The TSP replay signal is real but not monotone across offsets, while the ATSP compression effect remains visibly mixed.

5.2 Phase-6 TSP Replay Mechanism Study

Phase 6 asks why random replay beat raw failure replay in the initial TSP study. The follow-up campaign remains within symmetric TSPLIB TSP, but expands the mechanism space to curated and diagnostic replay variants, including diversity-aware failure replay, residual-failure replay, and compression applied to the stronger replay arms.

Phase 6 does not support the simple explanation that broader replay coverage alone accounts for the phase-5 TSP outcome. The best mean arm is diversity-aware failure replay, not random replay, and the archive diagnostics show that hardness tracks final held-out TSPLIB gap more strongly than descriptor diversity does.

Table 5.2 gives the most important mechanism comparisons.

The diagnostic side of phase 6 helps resolve the interpretation. Archive diversity has near-zero association with held-out TSPLIB gap (Pearson $r = -0.009478$, Spearman $\rho = -0.041013$), while archive hardness has a materially stronger positive association with worse held-out gap (Pearson $r = 0.39687$, Spearman $\rho = 0.371779$). The mechanism story is therefore not that random replay won because it maximized diversity in any simple sense. A more defensible reading is that naive failure replay concentrated too heavily on hard cases, while curated replay could recover some of that loss without producing a decisive universal win.

Comparison	Transfer delta	95% interval	Reading
random replay vs. no replay	−0.003123	[−0.027964, 0.021061]	slight mean advantage for random replay, but no decisive separation
raw failure replay vs. random replay	+0.001361	[−0.022536, 0.025457]	naive raw-failure replay does not recover the phase-5 TSP win
diversity-aware failure replay vs. raw failure replay	−0.008637	[−0.035972, 0.017862]	curation improves the mean arm, but the gain is not decisive at the paired level
residual-failure replay vs. raw failure replay	+0.022578	[−0.005643, 0.04867]	residual-failure replay underperforms the naive raw-failure arm
random replay with compression vs. random replay	+0.014874	not favorable	compression lowers novelty but worsens transfer on the random-replay arm
diversity-aware failure replay with compression vs. diversity-aware failure replay	+0.021526	not favorable	compression also hurts the best diversity-aware replay arm

Table 5.2: Key phase-6 TSP replay-mechanism comparisons, based on twenty paired offsets. Positive deltas are worse because the metric is lower-is-better transfer gap.

5.3 Implications of the Routing Phases

Phases 5 and 6 refine the dissertation's replay claim substantially.

Replay can help on benchmark routing, but the effect is conditional rather than universal. Raw failure replay is not a reliable default, and curation matters more than failure accumulation alone. Compression reduces novelty growth, but in the completed TSP mechanism study it does not improve the winning replay arms. The same replay logic also does not transfer cleanly across TSP, ATSP, and CVRP.

These findings remain methodologically important even though they are mixed. They prevent the thesis from over-claiming, and they motivate the later phases that change the search object rather than simply adding more replay variants to the same scaffold.

Chapter 6

Operator Discovery, Portfolio Control, and Real-World CVRP Studies

The later phases of the project deliberately raise the bar. Instead of asking only whether replay can help a fixed scaffold, they ask whether the search can discover reusable operators, learn an interpretable controller over a frozen portfolio, synthesize feasible solver logic on real-world CVRP instances, or survive explicit matched-budget controls on that same bounded CVRP regime. These phases distinguish bounded wins from claims the evidence cannot support.

6.1 Phase 7: Modular Operator Discovery

Phase 7 changes the search object from a whole TSP heuristic to a single modular operator. The official campaign contains twenty paired runs, 800 modular epochs, zero modular generation fallbacks, zero operator materialization fallbacks, and zero 'missing_build_operator' failures. The scientific question, however, is stricter than in the routing replay phases. A candidate operator is not counted as a discovery unless it survives transplant across multiple scaffolds, family-level evaluation, ablation, and Pareto-style practicality checks.

Table 6.1 summarizes the main outcome.

The distinction here is between *rediscovery* and *validated discovery*. Rediscovery means that similar operator families reappear across seeds, which suggests structured attractors in the search space. Validated discovery requires more: a reusable operator must show positive transplant evidence, family-level benefit, acceptable runtime behavior, and ablation evidence that the operator itself matters. No candidate met all of those criteria in the completed campaign.

Phase 7 is therefore a negative result on the retained endpoint, but it remains

Item	Result
Best overall condition	full-solver evolution, transfer gap 0.117916, TSPLIB gap 0.164668
Best modular condition	modular operator with random replay, transfer gap 0.118833, TSPLIB gap 0.230474
Modular evolution vs heuristic-only baseline	transfer delta −0.001715, 95% CI [−0.003418, 0.000085]; TSPLIB delta −0.003035, 95% CI [−0.010147, 0.000716]
Modular evolution vs full-solver evolution	TSPLIB delta +0.067355, 95% CI [0.040498, 0.094769] in favor of full-solver evolution
Validation outcome	no candidate satisfied the conservative surviving-operator rule
Rediscovery signal	56 distinct rediscovery signatures; the most common family was a double-bridge segment-reversal perturbation

Table 6.1: Phase-7 operator-discovery summary, based on twenty paired offsets. The search generates recurrent operator families, but no operator survives the full validation pipeline.

informative. It shows that the validation pipeline rejects brittle or scaffold-specific artifacts and prevents an unsupported reusable-operator claim. Appendix A.4 shows a representative rediscovered double-bridge perturbation artifact from this phase.

6.2 Phase 8: Adaptive Heuristic Portfolio Control

Phase 8 reframes the problem as algorithm selection. Instead of inventing low-level operators, the model evolves a bounded and interpretable controller over a frozen portfolio of known TSP heuristics. This target is more conservative. If phase 7 suggests that reusable low-level innovation is hard to validate, then a higher-level selection problem may still admit useful learning.

The completed campaign contains twenty paired runs, eight compared conditions, judge support in every run, and only three controller-generation errors across the whole replay-aware arm. All three of those errors occurred in rejected epochs, so no accepted final controller depends on fallback behavior.

Table 6.2 gives the key phase-8 comparisons.

The central structural result is the first row of interpretation in Table 6.2: the oracle selector ties the best single fixed heuristic on the primary held-out TSPLIB endpoint. In algorithm-selection terms, the frozen portfolio has almost no oracle headroom on that endpoint. If the oracle does not beat the single best fixed heuristic, then no learned controller should be expected to produce a strong positive selector-regret result there.

This structural constraint explains the mixed phase-8 outcome. The adaptive controller does learn something useful relative to a weak one-shot LLM rule baseline, but it does not beat the strongest fixed or supervised baselines on the primary endpoint. The limitation is not only controller learning. It is also the lack of complementary headroom in the frozen portfolio being controlled.

Comparison or statistic	Result
Best held-out TSPLIB condition	best single fixed heuristic, mean gap 0.07226
Oracle selector vs best fixed heuristic	held-out TSPLIB delta 0.0; transfer-gap delta −0.004544
Supervised selector vs best fixed heuristic	held-out TSPLIB delta 0.0; transfer-gap delta 0.0
LLM static selector vs supervised selector	held-out TSPLIB delta +0.121255, 95% CI [0.099278, 0.140226]
Adaptive controller vs LLM static selector	held-out TSPLIB delta −0.038118, 95% CI [−0.06868, −0.00707]
Replay-aware adaptive controller vs no-replay adaptive controller	held-out TSPLIB delta +0.010114, 95% CI [−0.020641, 0.041674]
Full-solver evolution vs adaptive controller	held-out TSPLIB delta +0.011853, 95% CI [−0.025032, 0.049041]

Table 6.2: Phase-8 adaptive-portfolio summary, based on twenty paired offsets. Lower deltas are better because the primary endpoint is held-out TSPLIB gap.

6.3 Phase 9: Real-World CVRP Solver Evolution

Phase 9 is the most realistic completed campaign in the dissertation. The search object is now a bounded whole solver for CVRPLIB Uchoa X instances [31]. The solver must return a full route set, satisfy customer-coverage and vehicle-capacity constraints, and tolerate stronger validator pressure than the earlier routing scaffolds.

The benchmark split used in the official bounded suite is listed in Table 6.3.

The official phase-9 campaign contains all four conditions in every paired run, judge support remains enabled throughout, and the accepted solvers do not rely on fallback code or hidden timeout failures. Across the whole campaign, the solver-evolution arm records 39 accepted epochs out of 160 total epochs, 0 generation fallbacks, 0 accepted errored generations, and full held-out feasibility in every official run.

Table 6.4 summarizes the main held-out outcomes.

The paired comparisons sharpen the claim. Solver evolution beats nearest-neighbor by −0.091204 on the held-out penalized gap, with 95% CI from −0.128136 to −0.055417. It also beats regret-insertion plus local search by −0.284160, with 95% CI from −0.321769 to −0.247706. Against Clarke–Wright, however, the sign reverses: solver evolution is worse by +0.108465, with 95% CI from 0.071184 to 0.144731.

Two additional properties matter for interpretation. First, the solver-evolution arm remains fully feasible on both train and held-out panels throughout the completed campaign. That means the loop preserves feasible executable solver updates under validator pressure rather than only generating low-gap candidates that fail validation. Second, the resulting solver set is not one repeated artifact. There are sixteen distinct final solver fingerprints across the twenty official runs, although some no-acceptance runs remain at the same starting incumbent.

The main limitation is runtime variability. The evolved solvers have mean

Training instances	Held-out instances
X-n101-k25, X-n110-k13, X-n125-k30, X-n134-k13, X-n157-k13, X-n214-k11	X-n176-k26, X-n190-k8, X-n223-k34, X-n247-k50, X-n275-k28

Table 6.3: Phase-9 bounded CVRPLIB X split, with six training instances and five held-out instances. The train/held-out separation is fixed before the campaign and is not reopened during solver evolution.

Condition	Held-out feasibility	Mean penalized gap	Reading
Clarke–Wright savings	1.0	0.073766	strongest fixed baseline
Solver evolution	1.0	0.182231	bounded positive result, but behind Clarke–Wright
Nearest-neighbor constructive	1.0	0.273435	clearly weaker baseline
Regret insertion plus local search	1.0	0.466391	weakest baseline on the primary endpoint

Table 6.4: Phase-9 held-out results on bounded real-world CVRP, based on twenty paired runs over the five held-out X instances. Lower penalized gap is better.

held-out runtime about 821.7 ms, median runtime about 255.2 ms, and a heavy upper tail above seven seconds in the worst run-level case. Phase 9 does not support a claim of practical dominance. It is a bounded real-world result: given only the specification, validator, scorer, and training instances, the loop can generate feasible held-out solver logic that improves over weaker baselines without surpassing the strongest fixed heuristic in the benchmark regime.

This runtime profile also clarifies the quality trade-off. The evolved solvers improve the primary held-out gap relative to the weaker constructive baselines while preserving feasibility, but they do so with materially less predictable wall-clock cost than the simpler fixed heuristics. In other words, the phase-9 result is best interpreted as evidence of bounded solution-quality adaptation under validator pressure, not as a ready-made runtime-efficient replacement for the strongest classical solver.

Figure 6.1 makes those boundary conditions explicit at the run level. The train-versus-held-out scatter shows that the stronger phase-9 runs stay strong off the training panel rather than collapsing catastrophically out of sample, but the held-out relationship remains noisy. The runtime panel shows a heavy-tailed cost profile, and the paired delta panels show why the phase-9 result remains bounded: solver evolution beats the weaker nearest-neighbor baseline in most matched runs, but it beats Clarke–Wright only in a small minority.

Figure 6.2 adds two diagnostics that are not visible in the final held-out mean alone. First, accepted updates are sparse and spread across the entire eight-epoch budget rather than concentrated at the start of the run. Second, the instance-level contrast is uniform in sign: averaged across the twenty official runs, solver evolution beats nearest-neighbor on all five held-out X instances, but it trails Clarke–Wright on all five.

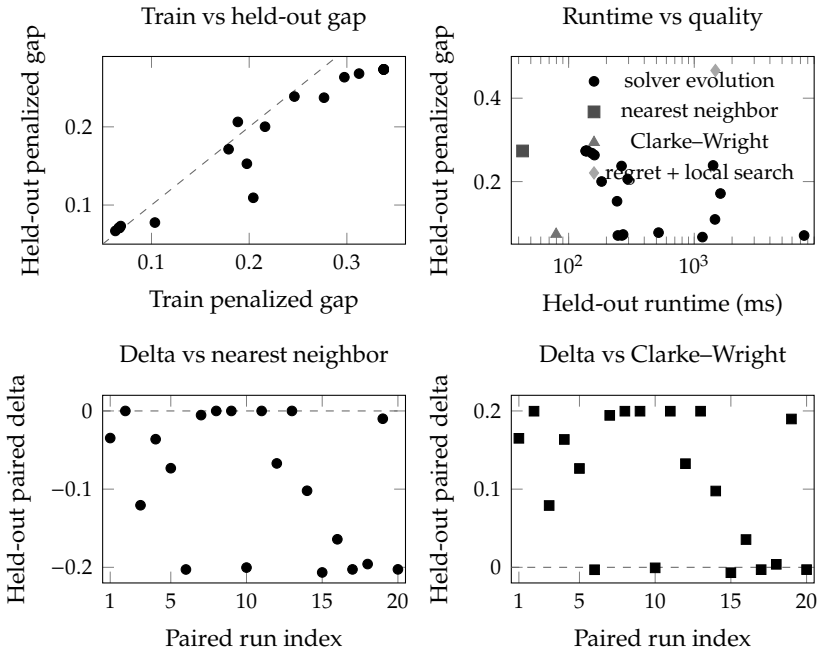

Figure 6.1: Diagnostic views for the official phase-9 solver-evolution campaign, using all twenty paired runs. The upper panels show that good train performance does not guarantee strong held-out performance and that runtime has a heavy upper tail. The lower panels show the paired held-out deltas against the two most important baselines. Negative values are better because the endpoint is lower-is-better penalized gap.

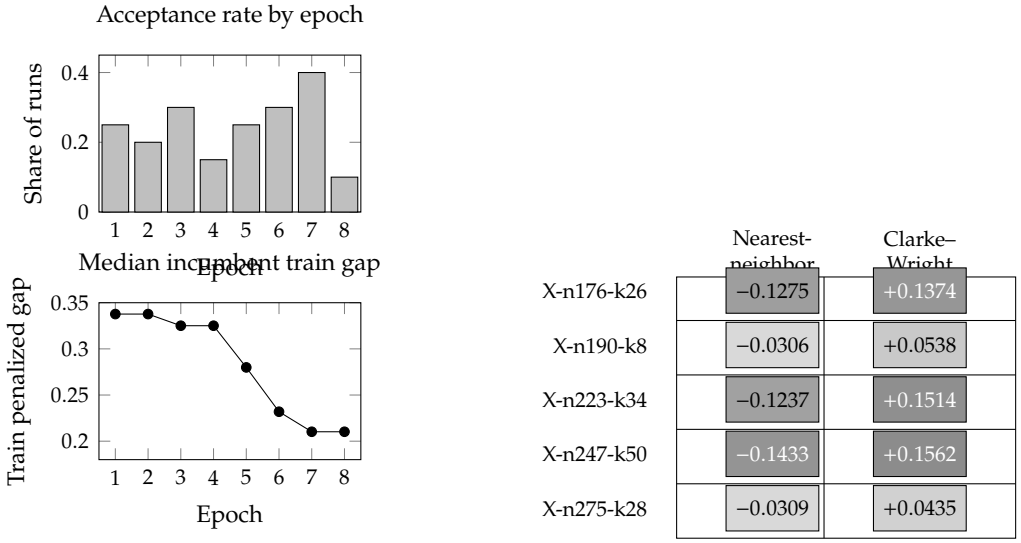

Figure 6.2: Additional diagnostics for the official phase-9 solver-evolution campaign. The left panels use all twenty official runs and track the accepted-update rate by epoch together with the median incumbent train penalized gap, starting from the shared nearest-neighbor constructive incumbent. The right panel aggregates the five held-out X instances across the same twenty runs. Solver evolution beats nearest-neighbor on every held-out instance, but it trails Clarke–Wright on every held-out instance. Darker cells mark larger absolute differences, and negative values favor solver evolution.

Appendix A.5 shows a representative accepted phase-9 solver fragment, and Appendix A.6 records a representative validator rejection. Those artifacts help explain both sides of the result: the loop can preserve feasible route-improvement logic, but validator pressure still filters out many large or syntactically salvaged candidates.

6.4 Phase 9 Closeout: Budget-Control CVRP Study

The original phase-9 campaign established that bounded solver evolution on real-world CVRP is feasible and stronger than the weakest fixed baselines, but it left one mechanism question open: was replay helping because it preserved useful incumbent structure, or simply because the loop was allowed to spend more candidate budget than a direct generation baseline? The closeout keeps the same bounded CVRPLIB X split, validator, and held-out endpoint, then introduces matched learned controls to answer that narrower question directly.

The official closeout campaign contains all six conditions in every paired run, judge support remains enabled throughout, and the three learned closeout arms record zero generation fallbacks and zero generation errors. Replay-aware solver evolution is also the only learned arm with full held-out feasibility in all twenty official runs. The budget-matched-no-replay arm remains feasible in only three runs, and the direct-generate-plus-one- repair arm remains feasible in only five.

Table 6.5 summarizes the held-out outcome.

The paired comparisons make the mechanism claim precise. Replay-aware solver evolution beats the budget-matched no-replay control by -2.761632 on the held-out penalized gap, with 95% CI from -3.369777 to -2.201586. It beats the direct-generate-plus-one-repair control by -5.115288, with 95% CI from -7.451608 to -2.946446. Against Clarke–Wright, however, the sign remains unfavorable: replay-aware solver evolution is worse by $+0.097348$, with 95% CI from 0.055994 to 0.138558.

The closeout addresses the mechanism question because the weaker learned arms fail in different ways. The budget-matched no-replay arm accepts more epochs than replay-aware solver evolution, yet it often produces code changes that do not preserve held-out feasibility. The bounded direct control occasionally produces a strong feasible solver, but it is unstable and usually collapses on the same held-out panel. These contrasts indicate that replay contributes more than extra candidate count alone: in this bounded regime it stabilizes the search around an incumbent well enough to dominate both learned controls, even though it still does not beat the strongest fixed Clarke–Wright baseline.

Condition	Held-out feasibility	Mean penalized gap	Reading
Clarke–Wright savings	1.0	0.073766	strongest fixed baseline, unchanged from the original phase-9 suite
Replay solver evolution	1.0	0.171114	only learned arm with full held-out feasibility under the closeout controls
Nearest-neighbor constructive	1.0	0.273435	weaker fixed baseline retained for continuity with phase 9
Regret insertion plus local search	1.0	0.466391	weakest fixed baseline on the primary endpoint
Budget-matched no replay	0.15	2.932746	memoryless learned control with the same overall candidate budget
Direct generate plus one repair	0.25	5.286402	bounded direct-synthesis control with one scripted repair step

Table 6.5: Held-out results for the official phase-9 closeout campaign, based on twenty paired runs over the same five held-out X instances used in the original phase-9 suite. Lower penalized gap is better.

6.5 Interpretive Pattern Across the Later Phases

Phases 7 through the phase-9 closeout share a common pattern. Once the search object or validator becomes more demanding, the loop continues to produce executable diversity, but the surviving positive claims become narrower.

Phase 7 shows rediscovery without validated reusable operator discovery. Phase 8 shows adaptive learning over a weak one-shot LLM baseline, but not over the strongest fixed or supervised selectors when selector headroom is absent. Phase 9 shows feasible real-world solver evolution that beats weak baselines, but not the best fixed baseline. The phase-9 closeout shows that replay-aware incumbent-preserving search has a measurable advantage over matched learned controls, yet still not enough to displace the strongest fixed classical baseline.

Taken as a whole, these later phases motivate the capability-bounded interpretation adopted in the dissertation. The retained evidence still includes non-zero held-out improvement under some conditions, yet the later phases repeatedly block the stronger claim that the system is already discovering dominant general-purpose optimization methods.

Chapter 7

Summary, Conclusions, and Future Work

The dissertation addressed a narrow question: can an LLM-guided code-evolution loop generate useful heuristic adaptation under controlled evaluation? The completed evidence supports a limited positive answer. Across several families, the loop produces executable diversity and sometimes preserves useful refinements. The same evidence also shows that the resulting gains are strongly bounded by validator pressure, baseline strength, benchmark headroom, and the exact search object under evolution.

7.1 Cross-Family Synthesis

The completed phases reveal recurring patterns rather than one isolated success or failure. Those patterns are best summarized at two scales. Table 7.1 is a master evidence atlas of the retained empirical record. Table 7.2 then distills the recurring cross-family patterns that survive that evidence.

Table 7.1: Master evidence atlas across the completed dissertation phases. "Baseline / control" names the phase-specific comparator used for the reported main delta. Lower deltas are better. Cells are marked "—" when the phase does not admit one matched paired contrast without forcing an artificial comparison.

Phase	Family	Search object	Conditions	N / pairing	Primary endpoint	Best LLM condition	Baseline / control	Main delta	95% CI	Interpretation
1–4	simple games	full policy code	3 envs, 3 reviewed recipes	10 transfer runs	held-out win rate, margin	territory control, win rate 0.9467, margin 34.17	—	—	—	Held-out transfer is measurable, but novelty spikes still collapse into churn often enough that code novelty alone is not evidence of adaptation.
5	TSP	bounded heuristic code	4 replay arms	20 paired offsets	held-out TSPLIB gap	random replay, mean 0.152211	no replay, mean 0.183325	−0.0311	[−0.0602, −0.0008]	Clearest replay win in the thesis; the best arm is random replay rather than raw failure replay.
5	ATSP	bounded heuristic code	4 replay arms	20 paired offsets	held-out ATSP gap	compressed failure replay, mean 0.187225	no replay, mean 0.192847	−0.0056	[−0.0191, 0.0071]	Compression-aware replay is mildly favorable, but the primary endpoint is not cleanly separated under paired uncertainty.
5	CVRP	bounded heuristic code	4 replay arms	20 paired offsets	held-out CVRPLIB gap	no replay, mean 0.235779	random replay, mean 0.246032	−0.0076	[−0.0146, −0.0006]	Replay is not a stable default even within routing; the strongest family-level condition in the replicated extension is the no-replay arm.
6	TSP	bounded heuristic code	7 replay arms	20 paired offsets	held-out TSPLIB gap	diversity-aware failure replay	raw failure replay	−0.008637	[−0.035972, 0.017862]	Replay curation matters more than archive size alone; archive hardness tracks worse held-out transfer more strongly than descriptor diversity does.

Continued on the next page

Table 7.1 continued from the previous page

Phase	Family	Search object	Conditions	N / pairing	Primary endpoint	Best LLM condition	Baseline / control	Main delta	95% CI	Interpretation
7	TSP	modular operator code	7 conditions	20 paired offsets	held-out TSPLIB gap, transplant, ablation	full-solver evolution, TSPLIB 0.164668	heuristic-only baseline	−0.003035	[−0.010147, 0.000716]	Rediscovery signatures recur, but no operator survives the full validation pipeline strongly enough to support a reusable-operator claim.
8	TSP	controller over frozen portfolio	8 conditions	20 paired offsets	held-out TSPLIB gap, selector regret	adaptive controller, mean 0.155397	static LLM selector	−0.038118	[−0.06868, −0.00707]	Adaptive control beats a weak one-shot selector, but the fixed heuristic remains best overall because oracle headroom is near zero on the primary endpoint.
9	CVRP	bounded whole-solver code	4 conditions	20 paired offsets	held-out penalized gap	solver evolution, mean 0.182231	Clarke–Wright, mean 0.073766	+0.108465	[0.071184, 0.144731]	Feasible solver evolution beats weaker baselines, but it does not surpass the strongest fixed heuristic and it carries a heavy runtime tail.
9 close-out	CVRP	bounded whole-solver code	6 conditions, matched learned controls	20 paired offsets	held-out penalized gap	replay solver evolution, mean 0.171114	budget-matched no replay, mean 2.932746	−2.761632	[−3.369777, −2.201586]	A replay-specific advantage remains after matched learned controls are added, but the strongest fixed Clarke–Wright baseline still remains best overall.

Evidence family	Strongest positive signal	Main limiting factor	Dominant failure mode
Simple games	measured behavioral transfer, especially in territory control	environment dependence	lexical novelty without robust behavioral gain
Routing replay (phases 5–6)	replay helps on symmetric TSP	family-specific mechanism sensitivity	unstable transfer of replay recipes across TSP, ATSP, and CVRP
Operator discovery (phase 7)	recurring operator rediscovery signatures	strict transplant and ablation criteria	scaffold-specific or brittle operators
Adaptive portfolio (phase 8)	adaptive controller beats weak one-shot LLM selector	near-zero oracle headroom on the primary endpoint	lack of complementarity in the frozen portfolio
Real-world CVRP (phase 9)	feasible solver evolution beats weaker baselines	strong Clarke–Wright baseline and runtime variability	bounded improvement without strongest-baseline dominance
Real-world CVRP closeout (phase 9 closeout)	replay-aware search beats both matched learned controls	strong Clarke–Wright baseline still remains best overall	memoryless or direct learned controls collapse on feasibility and gap

Table 7.2: Recurring patterns across the completed phases. The positive signals are real, but each is paired with a concrete limiting factor rather than with an unrestricted win narrative.

The atlas separates three things that are easy to conflate in a code-evolution project: whether the loop can generate executable diversity at all, whether that diversity survives held-out evaluation, and whether the resulting gain remains visible once the strongest available baseline is used as the reference point.

In the archival phase chronology, the atlas covers the retained evidence from phases 1 through 10 in bundled form, together with the scoped phase-9 closeout on the same bounded CVRP regime. The first row aggregates the simple-game record from phases 1, 2, 2b, 3, and 4; phases 5 through 9 appear directly; the closeout records the final matched-budget mechanism study on real-world CVRP; and phase 10 is the synthesis step used to interpret those retained results rather than a new benchmark family. The scoped phase-11 model-strength factorial remains outside the retained dissertation evidence base.

Four synthesis statements follow. Syntactic code novelty is easy for the loop to produce, but novelty is not sufficient for functional gain. Improvements are easiest to obtain when the benchmark leaves reachable headroom and when the baseline is not already very strong. The dominant failure mode changes by family: churn in games, replay-selection sensitivity in routing, validation collapse for modular discovery, selector-headroom limits in portfolios, and feasibility/runtime pressure in real-world CVRP. The most conservative interpretation consistent with the retained evidence is capability-bounded adaptation rather than general algorithm discovery.

The direct single-shot Codex CVRP baseline reinforces this final point. Evaluated descriptively through the same phase-9 validator, the interactive Codex-authored solver reaches full held-out feasibility with mean penalized gap 0.06778, which is close to the Clarke–Wright baseline and well ahead of both the replicated autonomous solver-evolution mean and the bounded direct closeout control. That comparison

is descriptive only, not a matched experimental condition, but together with the closeout it is consistent with the possibility that current interactive coding assistance can extract more stable value from the model in this regime than the bounded autonomous protocols retained in the dissertation.

7.2 Assessment of the Working Hypotheses

Table 7.3 revisits the hypotheses stated in Section 1.3.

This hypothesis-level summary captures the main contribution of the dissertation. The project does not end with a single dominant heuristic recipe. Instead, it leaves a more precise empirical account of where the loop transfers, where it weakens, and what kinds of controls are needed to separate those cases.

Hyp.	Status	Evidence-based assessment
H1	Supported	The loop generates executable diversity and produces useful held-out adaptation in the simple games, positive replay transfer on symmetric TSP, and feasible bounded improvement on real-world CVRP.
H2	Partially supported	Replay helps in some settings, and the phase-9 closeout shows a clear replay-specific advantage over matched learned controls. Even so, no single replay recipe dominates across the retained families, and the strongest fixed baselines still prevent a universal replay win claim.
H3	Supported	As validators strengthen and classical baselines consume more benchmark structure, the gains narrow. This pattern is strongest in phases 8 and 9, where the best fixed or supervised baselines remain ahead on the primary endpoint.
H4	Supported	Transfer exists, but it is partial and family-dependent. The strongest early transfer is concentrated in territory control, and the routing phases show that signals learned on one family do not carry over uniformly to others.

Table 7.3: Assessment of the working hypotheses. The evidence supports adaptation, but in a bounded and family-sensitive form.

7.3 Limitations

Several limitations should remain explicit.

1. The routing phases rely on bounded scaffolds and interfaces. This choice improves interpretability, but it also narrows the search space relative to unconstrained program synthesis.
2. The completed campaigns use fixed benchmark subsets rather than the full diversity of every public benchmark library.
3. The direct Codex CVRP baseline is descriptive and human-guided rather than replicated and randomized.
4. The dissertation does not retain a completed crossed experiment that cleanly separates base model tier from evolution technique under matched budgets. The phase-9 closeout resolves the replay-versus-budget question within one fixed model tier, but cross-tier attribution remains open.
5. Some family-level comparisons are qualitative by necessity because the primary metrics differ across games, TSP/ATSP, and CVRP.

These limitations do not invalidate the central claims. They define the boundary of those claims and help explain why the final interpretation is intentionally conservative.

7.4 Future Work

One possible next step is a clean model-strength continuum study. A suitable design would cross task family, model tier, and evolution technique under matched

candidate budgets, with at least single-shot, no-replay, and replay-based arms. Such a study would answer whether the gains observed here come mainly from the base model, from the outer search loop, or from their interaction.

Three additional directions follow directly from the retained results.

1. Replay should be revisited through curation rather than through larger archives alone. Phase 6 indicates that hardness and coverage matter more than simple diversity counts.

2. Adaptive portfolio work should begin with stronger portfolio complementarity. If the oracle selector cannot beat the single best heuristic on the primary endpoint, then controller learning has little room to show a clear gain.

3. Real-world solver evolution should be extended to harder routing families, stronger external references, and explicit runtime-quality trade-off analysis. CVRPTW and broader CVRPLIB regimes are natural candidates once the bounded X-instance setup is exhausted.

A further methodological direction is to formalize the direct interactive-coding baseline. The current Codex-authored CVRP solver is useful descriptively, but a frozen and replicated human-in-the-loop protocol would allow a fairer comparison between autonomous and interactive use of the same model class.

7.5 Final Conclusion

LLM-guided code evolution is scientifically worth studying because it can generate executable heuristic variation and, in some settings, preserve useful adaptation across nontrivial domains. At the same time, the completed evidence does not support a claim of broad autonomous solver discovery. The dissertation's main contribution is a cross-family methodology and evidence base for distinguishing useful heuristic evolution from code churn, and bounded progress from overstatement. That is a narrower claim than early enthusiasm around LLM-driven algorithm discovery often suggests, but it is the stronger and more defensible scientific result.

Appendix A

Representative Evolved Artifacts and Failure Modes

This appendix presents short excerpts from archived artifacts so that the empirical claims in the main chapters are not purely statistical. The excerpts are intentionally brief. They are included to show the kinds of changes the loop actually produced, not to replace the aggregate evidence with anecdotes.

A.1 Successful Simple-Game Adaptation

Listing A.1 comes from a replay-aware pursuit/evasion run that was retained as a functional adaptation in the phase-4 novelty audit. The main structural change is the opponent-model helper `opp_best_after`, which scores each candidate move against the opponent's likely reply rather than against the opponent's current position alone.

Listing A.1: Accepted pursuit/evasion policy excerpt from the replay-aware transfer suite.

```
def opp_best_after(selfx, selfy):
    opp_opts = neighbors(ox, oy)
    if not opp_opts:
        return ox, oy
    best = None
    bestv = None
    for px, py in opp_opts:
        v = d2(selfx, selfy, px, py)
        if opp_evader:
            if best is None or v > bestv:
                best, bestv = (px, py), v
        else:
            if best is None or v < bestv:
                best, bestv = (px, py), v
    return best

```

```
if self_evader:
    for nx, ny in self_moves:
        px, py = opp_best_after(nx, ny)
        v = d2(nx, ny, px, py)
        if bestv is None or v > bestv:
            bestv, best_move = v, (nx, ny)
```

A.2 Novel but Unhelpful Churn

Listing A.2 is a representative novelty-spike artifact from the replay-aware territory-control audit. It is structurally coherent and visibly different from the incumbent, but the archived selection record marks it as rejected because its score regressed from 50.5 to 39.5 against the same opponent.

Listing A.2: Rejected territory-control novelty spike from the replay-aware transfer audit.

```
if (nx, ny) in ot:
    sc += 18.0
elif (nx, ny) in unq:
    sc += 10.0
elif (nx, ny) in st:
    sc += 6.0
else:
    sc += 4.0

dsc = man(nx, ny, cx, cy)
sc += -0.7 * dsc
sc += 0.35 * man(ox, oy, cx, cy)

dso = man(nx, ny, ox, oy)
sc += -0.08 * dso
```

A.3 Replay-Generated TSP Heuristic Fragment

Listing A.3 shows a replay-generated TSP heuristic fragment from the phase-5 symmetric TSPLIB suite. The evolved object is not a full solver rewrite. It is a bounded configuration over construction, local-search, perturbation, restart, and acceptance modules.

Listing A.3: Phase-5 replay-generated TSP heuristic fragment.

```
def build_heuristic():
    return {
        "construction": {
            "seed_mode": "nearest_centroid",
            "candidate_limit": 28,
            "lookahead_limit": 4,
```

```
            "distance_weight": 1.05,
            "regret_bonus": 0.45,
        },
        "local_search": {
            "two_opt_passes": 8,
            "use_three_opt": True,
            "three_opt_samples": 10,
        },
        "perturbation": {
            "enabled": True,
            "mode": "segment_reversal",
            "strength": 2,
        },
        "acceptance": {
            "mode": "annealed",
            "worse_acceptance_threshold": 0.012,
        },
    }
```

A.4 Rediscovered Operator Family

Listing A.4 is a phase-7 perturbation operator that exemplifies the double-bridge and segment-reversal family highlighted in the rediscovery analysis. The phase-7 claim is not that this operator survived as a reusable component, but that related perturbation families reappeared repeatedly without clearing the full validation bar.

Listing A.4: Phase-7 perturbation operator from the rediscovered double-bridge family.

```
def build_operator():
    return {
        "name": "perturbation_two_cluster_bottleneck_escape_double_bridge_compact",
        "description": "Deterministic escape schedule for 2-opt stagnation.",
        "operator_type": "perturbation",
        "primary_mode": "double_bridge",
        "secondary_mode": "segment_reversal",
        "stagnation_threshold": 3,
        "strength": 3,
        "attempts": 2,
        "escalation_strength": 2,
    }
```

A.5 Bounded CVRP Solver Evolution

Listing A.5 shows a short excerpt from an accepted phase-9 CVRP solver. The fragment is representative of the later successful solvers: a deterministic Clarke–Wright construction remains visible, but it is followed by bounded route-improvement logic rather than by a simple constructive pass alone.

Listing A.5: Accepted phase-9 CVRP solver fragment.

```
def improve_swap(rs):
    if len(rs) < 2:
        return False
    best_delta = 0
    best_move = None

    for i in range(R - 1):
        ri = rs[i]
        for j in range(i + 1, R):
            rj = rs[j]
            for p in range(len(ri)):
                a = ri[p]
                for q in range(len(rj)):
                    b = rj[q]
                    if li - demands[a] + demands[b] > capacity:
                        continue
                    if lj - demands[b] + demands[a] > capacity:
                        continue
                    new_i = ri[:]
                    new_j = rj[:]
                    new_i[p] = b
                    new_j[q] = a
```

A.6 Rejected Validator Case

The failure modes are as important as the successes. Listing A.6 records a phase-9 rejection in which a large candidate was salvaged syntactically but still violated the solver-output contract. This kind of example explains why the thesis emphasizes validator pressure throughout the later chapters.

Listing A.6: Phase-9 rejection record with validator failure.

```
"accepted": false,
"errors": [
  "solver did not return a list of routes"
],
"generation_fallback_used": false,
"salvage_attempted": true,
"validation_issues": [
  "salvaged by trimming incomplete trailing block"
]
```

Bibliography

[1] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, *Deep reinforcement learning at the edge of the statistical precipice*, in Advances in Neural Information Processing Systems 35, 2021.

[2] M. Alissa, E. Özcan, and G. Kendall, *Automated algorithm selection: from feature-based to feature-free approaches*, Journal of Heuristics, 29 (2023), pp. 1–38.

[3] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, *Program synthesis with large language models*, arXiv preprint arXiv:2108.07732, (2021).

[4] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, *Neural combinatorial optimization with reinforcement learning*, arXiv preprint arXiv:1611.09940, (2016).

[5] Y. Bengio, A. Lodi, and A. Prouvost, *Machine learning for combinatorial optimization: A methodological tour d'horizon*, European Journal of Operational Research, 290 (2021), pp. 405–421.

[6] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, *Curriculum learning*, in Proceedings of the 26th International Conference on Machine Learning, 2009, pp. 41–48.

[7] M. Brittain, J. Bertram, X. Yang, and P. Wei, *Prioritized sequence experience replay*, arXiv preprint arXiv:1905.12726, (2019).

[8] E. K. Burke, M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and R. Qu, *Hyper-heuristics: A survey of the state of the art*, Journal of the Operational Research Society, 64 (2013), pp. 1695–1724.

[9] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain,

W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, *Evaluating large language models trained on code*, arXiv preprint arXiv:2107.03374, (2021).

[10] G. Clarke and J. W. Wright, *Scheduling of vehicles from a central depot to a number of delivery points*, Operations Research, 12 (1964), pp. 568–581.

[11] H. Dai, E. B. Khalil, Y. Zhang, B. Dilkina, and L. Song, *Learning combinatorial optimization algorithms over graphs*, in Advances in Neural Information Processing Systems 30, 2017.

[12] T. Dökeroglu, T. Kucukyilmaz, and E.-G. Talbi, *Hyper-heuristics: A survey and taxonomy*, Computers & Industrial Engineering, 187 (2024), p. 109815.

[13] A. Graves, M. G. Bellemare, J. Menick, R. Munos, and K. Kavukcuoglu, *Automated curriculum learning for neural networks*, in Proceedings of the 34th International Conference on Machine Learning, 2017, pp. 1311–1320.

[14] K. Helsgaun, *An effective implementation of the lin-kernighan traveling salesman heuristic*, European Journal of Operational Research, 126 (2000), pp. 106–130.

[15] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, *Deep reinforcement learning that matters*, Proceedings of the AAAI Conference on Artificial Intelligence, 32 (2018).

[16] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, *LiveCodeBench: Holistic and contamination free evaluation of large language models for code*, arXiv preprint arXiv:2403.07974, (2024).

[17] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, *A survey on large language models for code generation*, arXiv preprint arXiv:2406.00515, (2024).

[18] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. S. Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, *Competition-level code generation with AlphaCode*, Science, 378 (2022), pp. 1092–1097.

[19] S. Lin and B. W. Kernighan, *An effective heuristic algorithm for the traveling-salesman problem*, Operations Research, 21 (1973), pp. 498–516.

[20] S. Liu, C. Chen, X. Qu, K. Tang, and Y.-S. Ong, *Large language models as evolutionary optimizers*, in 2024 IEEE Congress on Evolutionary Computation (CEC), 2024, pp. 1–8.

[21] D. J. Mankowitz, A. Michi, A. Zhernov, M. Gelmi, M. Selvi, C. Paduraru, E. Leurent, S. Iqbal, J.-B. Lespiau, A. Ahern, T. Köppe, K. Millikin, S. Gaffney, T. Rashid, Y. Li, D. B. Dohan, H. Pham, D. Silver, K. Kavukcuoglu, P. Kohli, and O. Vinyals, *Faster sorting algorithms discovered using deep reinforcement learning*, Nature, 618 (2023), pp. 257–263.

[22] J.-B. Mouret and J. Clune, *Illuminating search spaces by mapping elites*, arXiv preprint arXiv:1504.04909, (2015).

[23] J. K. Pugh, L. B. Soros, and K. O. Stanley, *Quality diversity: A new frontier for evolutionary computation*, Frontiers in Robotics and AI, 3 (2016), p. 40.

[24] G. Reinelt, *TSPLIB—a traveling salesman problem library*, ORSA Journal on Computing, 3 (1991), pp. 376–384.

[25] J. R. Rice, *The algorithm selection problem*, Advances in Computers, 15 (1976), pp. 65–118.

[26] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, *Mathematical discoveries from program search with large language models*, Nature, 625 (2024), pp. 468–475.

[27] F. D. Ros, M. Soprano, L. D. Gaspero, and K. Roitero, *Large language models for combinatorial optimization: A systematic review*, arXiv preprint arXiv:2507.03637, (2025).

[28] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, *Prioritized experience replay*, arXiv preprint arXiv:1511.05952, (2015).

[29] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, *Mastering the game of go without human knowledge*, Nature, 550 (2017), pp. 354–359.

[30] P. Soviany, R. T. Ionescu, P. Rota, and N. Sebe, *Curriculum learning: A survey*, International Journal of Computer Vision, 130 (2022), pp. 1526–1565.

[31] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, and A. Subramanian, *New benchmark instances for the capacitated vehicle routing problem*, European Journal of Operational Research, 257 (2017), pp. 845–858.

[32] T. Vidal, *Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood*, Computers & Operations Research, 140 (2022), p. 105643.

[33] D. Zan, B. Chen, F. Zhang, D. Lu, B. Wu, B. Guan, W. Yongji, and J.-G. Lou, *Large language models meet NL2Code: A survey*, in Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 7443–7464.